



Riza Satria Perdana, S.T., M.T.

Teknik Informatika - STEI ITB

Exceptions

Exceptions

Pemrograman Berorientasi Objek

Exceptions

- Exceptional Event
- Definisi: sebuah event yang terjadi saat eksekusi program, yang mengganggu alur normal dari instruksi program
- Ketika sebuah error terjadi pada sebuah method, maka method tersebut akan menciptakan sebuah objek dan dilempar ke runtime system

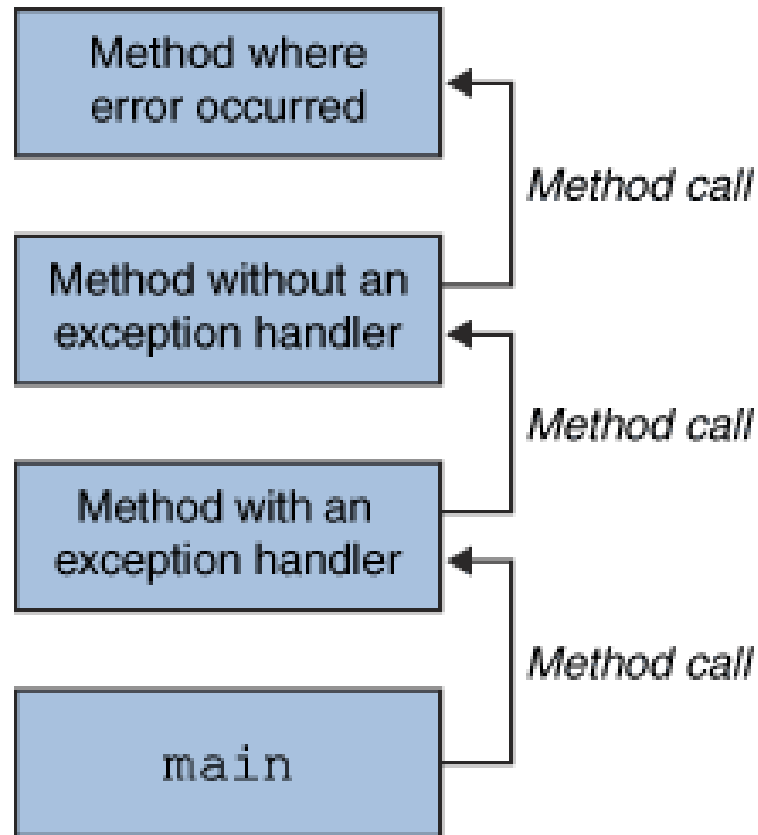
Exceptions

- Objek yang diciptakan itu disebut *exception object*
- Exception object mengandung informasi tentang error tersebut (termasuk tipe dan state program ketika error terjadi)
- Menciptakan exception object dan melempar ke runtime system disebut *throwing an exception*

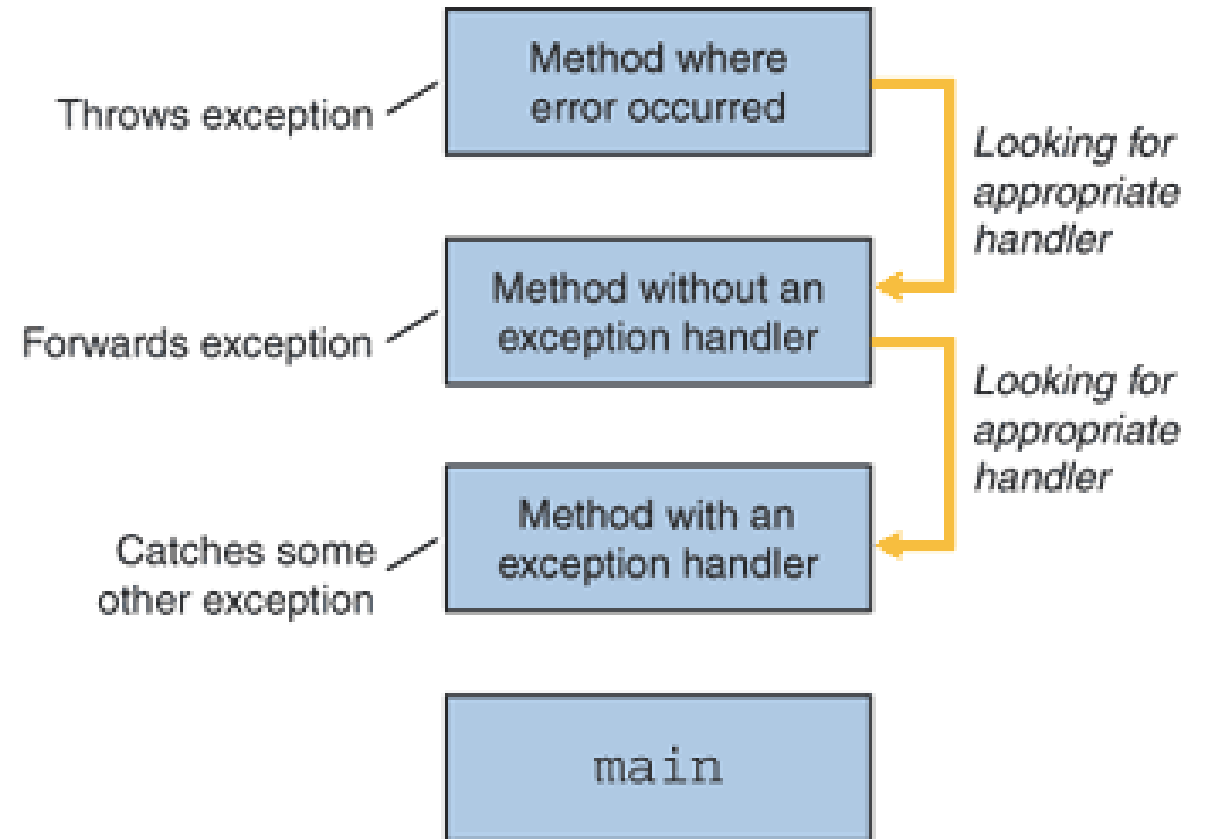
Exceptions

- Setelah method melempar exception, runtime system akan mencari sesuatu untuk meng-handle itu (disebut *exception handler*)
- Exception handler akan melakukan *catch the exception*

Exceptions Handler



The call stack.



Searching the call stack for the exception handler.

Exceptions Handler

- Komponen: try block, catch block, finally block

```
try {  
  
} catch (ExceptionType name) {  
  
} catch (ExceptionType name) {  
  
}
```

Terima Kasih



Riza Satria Perdana, S.T., M.T.

Teknik Informatika - STEI ITB

Exceptions

Exceptions Handler

Pemrograman Berorientasi Objek

Try Block

```
private Vector vector;  
private static final int SIZE = 10;  
  
PrintWriter out = null;  
  
try {  
    System.out.println("Entered try statement");  
    out = new PrintWriter(new FileWriter("OutFile.txt"));  
    for (int i = 0; i < SIZE; i++) {  
        out.println("Value at: " + i + " = "  
                    + vector.elementAt(i));  
    }  
}  
  
// catch and finally statements . . .
```

Try Block

- Berisi kode yang mungkin memunculkan exception
- Try block bisa dibuat untuk setiap kode yang mungkin menimbulkan exception
- Bisa juga dengan mengumpulkan banyak kode dalam sebuah try block

Catch Block

- Berisi kode yang merupakan exception handler
- Menangani exception dengan tipe yang sesuai dengan tipe yang ditunjukkan pada argumen

Catch Block

```
try {  
  
} catch (FileNotFoundException e) {  
    System.err.println("FileNotFoundException: "  
                        + e.getMessage());  
    throw new SampleException(e);  
  
} catch (IOException e) {  
    System.err.println("Caught IOException: "  
                        + e.getMessage());  
}  
}
```

Finally Block

- Berisi kode yang pasti akan dieksekusi setelah try block
- Finally block akan dieksekusi baik terjadi exception maupun tidak

Finally Block

```
finally {  
    if (out != null) {  
        System.out.println("Closing PrintWriter");  
        out.close();  
    } else {  
        System.out.println("PrintWriter not open");  
    }  
}
```



Kode Lengkap

```
public void writeList() {
    PrintWriter out = null;

    try {
        System.out.println("Entering try statement");
        out = new PrintWriter(new FileWriter("OutFile.txt"));
        for (int i = 0; i < SIZE; i++)
            out.println("Value at: " + i + " = " + vector.elementAt(i));

    } catch (ArrayIndexOutOfBoundsException e) {
        System.err.println("Caught " + "ArrayIndexOutOfBoundsException: " + e.getMessage());
    } catch (IOException e) {
        System.err.println("Caught IOException: " + e.getMessage());
    } finally {
        if (out != null) {
            System.out.println("Closing PrintWriter");
            out.close();
        }
        else {
            System.out.println("PrintWriter not open");
        }
    }
}
```



Terima Kasih



Riza Satria Perdana, S.T., M.T.

Teknik Informatika - STEI ITB

Exceptions

Exception Class

Pemrograman Berorientasi Objek

Cara Throw Exception

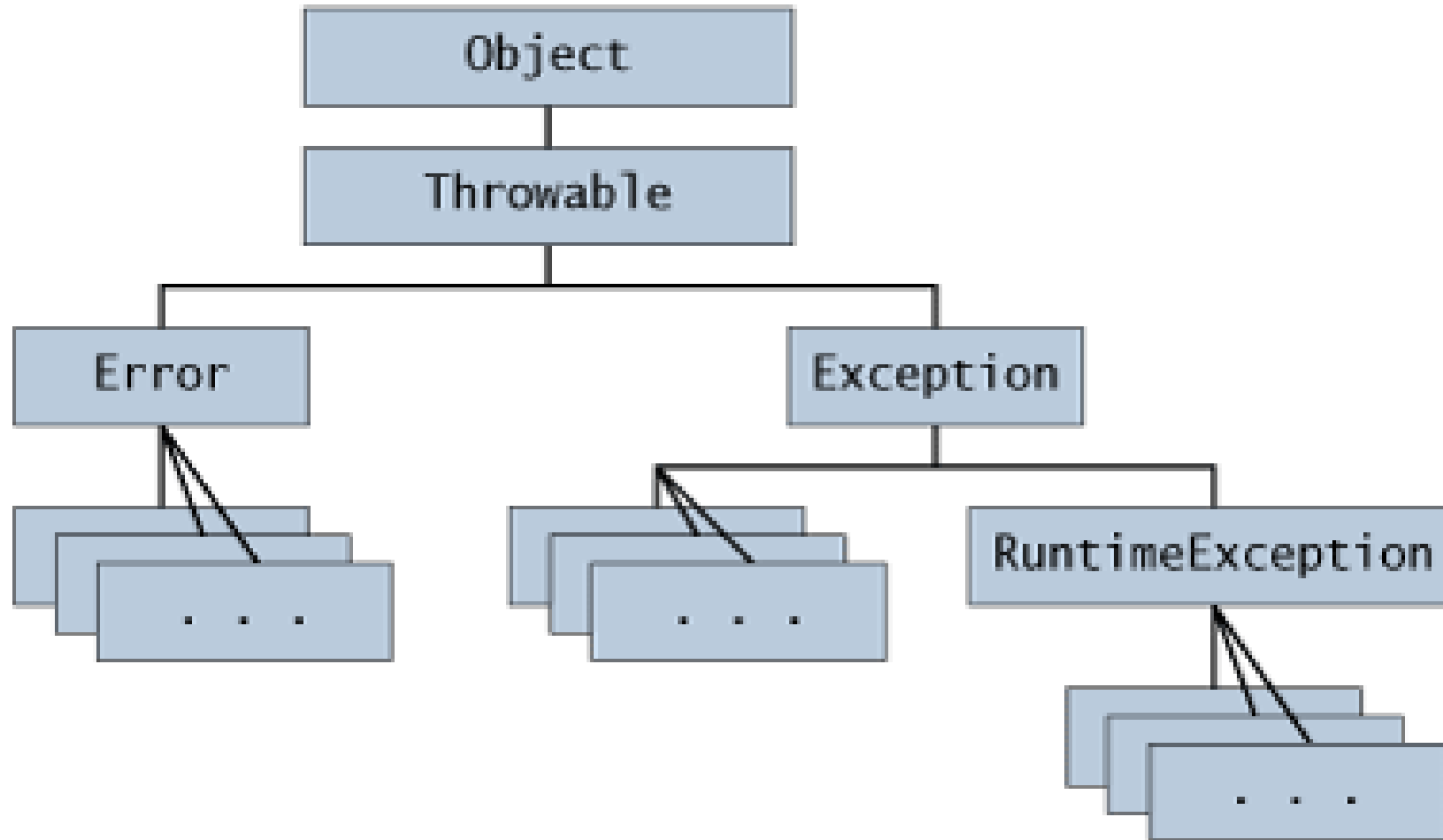
- Menggunakan **throw** statement
- Membutuhkan sebuah argumen berupa throwable objek
- Throwable objek adalah instansiasi dari kelas Throwable atau turunannya

throw *someThrowableObject*;

Throw

```
public Object pop() {  
    Object obj;  
  
    if (size == 0) {  
        throw new EmptyStackException();  
    }  
  
    obj = objectAt(size - 1);  
    setObjectAt(size - 1, null);  
    size--;  
    return obj;  
}
```

Throwable Class & Subclass



Chained Exception

- Method bisa merespon terjadinya exception dengan melempar exception lagi

```
try {  
  
} catch (IOException e) {  
    throw new SampleException("Other IOException", e);  
}
```

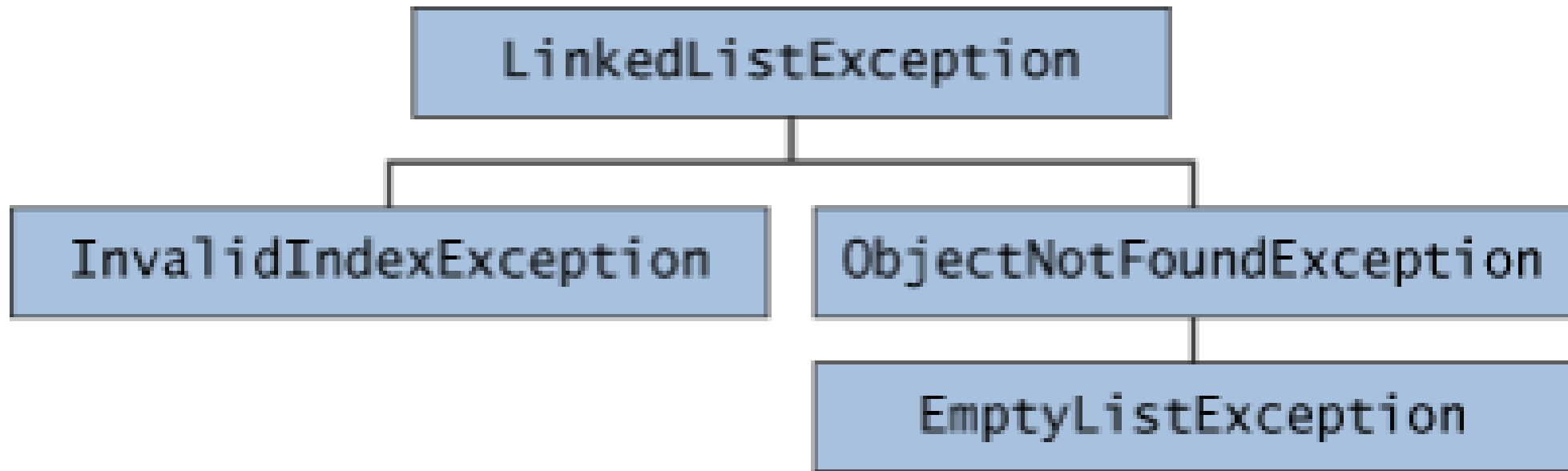
Membuat Exception Class

- Exception object dapat menggunakan kelas yang dibuat orang lain (termasuk yang ada di Java platform) atau membuat sendiri
- Exception Subclass dapat digunakan sebagai parent
- Tambahkan kata Exception di nama kelas untuk memudahkan

Membuat Exception Class

- Alasan membuat exception class sendiri:
 - Tidak ada di Java platform
 - Lebih memudahkan pengguna kelas
 - Digunakan di banyak tempat
- Perlu diperhatikan juga independensi paket atau *self-contained*

Contoh LinkedList Exception



Kelebihan Exception

- Memisahkan *error handling code* dengan *regular code*
- Mempropagasikan *error* ke atas (ke method pemanggil) di *call stack*
- Mengelompokkan dan membedakan tipe *error*

Terima Kasih